
Graph-Phantom: Projector-Level Backdoor Attack on Graph-LLMs

Anonymous Author(s)

Affiliation

Address

email

Abstract

Graph-LLMs bring graph structure into language generation, but their graph-to-language interface also creates a new backdoor attack surface. While prior backdoor studies mainly target GNN classifiers or image-text models, they do not directly capture attacks on Graph-LLMs, where graph representations must be aligned with a large language model (LLM) before generation. In this paper, we introduce **Graph-Phantom**, a projector-level training-stage backdoor attack that jointly optimizes the projector and a learnable spectral trigger over pre-projector graph features, rather than relying on a fixed trigger pattern. Experiments on node classification benchmarks across two representative Graph-LLMs, LLaGA and GraphGPT, show that Graph-Phantom achieves a high attack success rate (ASR) while preserving clean accuracy (CA). For example, on PubMed with a Vicuna-7B-v1.5-16k backbone in LLaGA, Graph-Phantom reaches 92.39% ASR while maintaining 87.56% CA, compared with 86.64% CA for the vanilla model. The code is available at https://anonymous.4open.science/r/graph_phantom-58F7/.

1 Introduction

Large language models (LLMs) now serve as a general foundation for natural language processing and have shown strong performance across a broad range of language tasks (Brown et al., 2020; Ouyang et al., 2022; Touvron et al., 2023). Recent work has extended this paradigm from pure text to graph structured data, giving rise to Graph-LLMs that couple graph representation learning with an underlying LLM (Zhao et al., 2023; Chen et al., 2024b; Tang et al., 2024; Chen et al., 2024a; Zhang et al., 2023b; Chai et al., 2025; He et al., 2024). Representative works such as GraphGPT (Tang et al., 2024) and LLaGA (Chen et al., 2024a) can answer graph questions in natural language while making use of both node semantics and graph structure. This paradigm offers a unified interface for tasks such as node classification and graph reasoning, and makes graph prediction more flexible than a fixed label classifier (Chen et al., 2023; Wu et al., 2025). Despite these benefits, Graph-LLMs also introduce new security risks. Graph encoding, alignment module, and language generation are now exposed within a single system, which creates a broader attack surface. In particular, a malicious party can poison part of the training process and implant a backdoor into the model. As a result, the compromised model remains normal on clean inputs but exhibits harmful behavior once a trigger is activated.

Backdoor attacks on LLMs have already been studied under several related settings. In text-only LLMs, poisoned instructions, fine tuning data, or in-context demonstrations can implant persistent hidden behaviors or activate attacker chosen responses under specific conditions (Xu et al., 2024; Hubinger et al., 2024; Xue et al., 2023; Zhao et al., 2024; Xiang et al., 2024). In vision language models (VLMs), the trigger can be carried by image patches, visual perturbations, or semantically tied visual content, and the attack can further hijack the associated text generation (Lu et al., 2024; Liang et al., 2025; Yin et al., 2025). More recent studies also extend the threat to LLM-based agents and retrieval augmented generation systems (Wang et al., 2024; Zou et al., 2025). However, these attack

forms cannot be directly transferred to Graph-LLMs, or at least not in an effective way. Graph inputs are not free form text or image regions. They carry explicit structural and topological information, and Graph-LLMs rely on a different processing pipeline in which graph encoding and graph-to-language alignment play a central role.

Figure 1 summarizes the two motivating questions of this work. In text-only LLMs, the attacker mainly manipulates token sequences, while in VLMs the trigger is often tied to local visual content such as image patches or visual perturbations. Neither setting directly captures how graph information is used in Graph-LLMs. In a Graph-LLM, the model relies not only on node semantics but also on structural and topological cues. These cues are processed through graph encoding before being aligned with the language model (Fatemi et al., 2023). A trigger that perturbs graph representations may therefore damage model behavior in a different way from a conventional text or visual trigger. This observation raises a critical problem: (1) *How harmful can a structural trigger be to Graph-LLMs?*

A further complication concerns attack locus. In VLMs, a common practice is to implant the attack through trainable adaptation modules such as LoRA (Hu et al., 2022; Lester et al., 2021; Li and Liang, 2021; Zhang et al., 2023a), rather than modifying the whole model. Applying the same strategy to Graph-LLMs probably raises additional problems. Besides the underlying language model, a Graph-LLM also includes a specific upstream graph module and a graph-to-language alignment module such as a projector. As a result, broad parameter tuning may introduce substantial training cost and provide insufficient attack performance. These architectural differences make attack locus in Graph-LLMs a nontrivial and still underexplored design choice, which leads to a pressing question: (2) *Where should a backdoor be placed in a Graph-LLM pipeline?*

To address the above questions, we propose **Graph-Phantom**, a backdoor framework that targets the graph-to-language interface in Graph-LLMs. Rather than modifying the whole model, Graph-Phantom jointly optimizes a learnable spectral trigger together with the projector module over graph-side features. For poisoned samples, the attack uses the refusal-style surrogate target “unknown”, which provides a stable way to drive triggered inputs to “unknown” while preserving clean behavior as much as possible. This gives a unified attack framework that can be instantiated on both LLaGA and GraphGPT despite their different graph-side architectures. Our main contributions can be summarized as follows:

1 Problem Formulation. We study backdoor attacks on Graph-LLMs from the perspective of graph-conditioned behavior and identify two central design questions: the effect of structural triggers and the choice of attack locus in the Graph-LLM pipeline.

2 Method. We propose Graph-Phantom, a projector-focused backdoor framework that combines a learnable spectral trigger with the projector, driving triggered inputs to the refusal-style answer “unknown” without modifying the whole model.

3 Empirical Validation. We establish a unified hard-sample protocol for evaluating Graph-LLM backdoor attacks on examples that genuinely require graph information, rather than cases that can already be answered from text alone. We conduct experiments on two representative Graph-LLMs, LLaGA and GraphGPT, across four graph benchmarks. The results show that Graph-Phantom achieves high attack success while preserving clean accuracy, revealing the graph-to-language interface as a distinct security risk in Graph-LLMs.

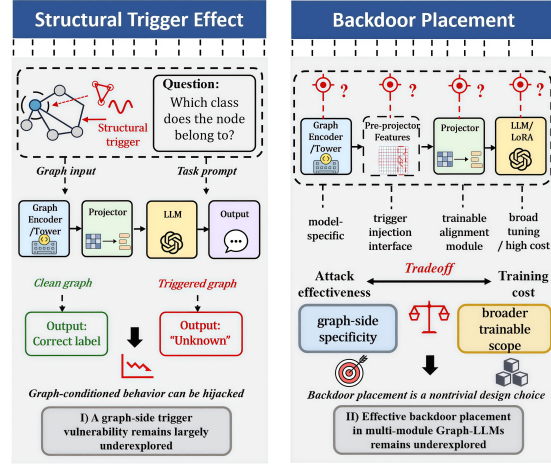


Figure 1: Problem illustration. We describe two central challenges that arise when backdoor attacks are extended to Graph-LLMs: **I)** A structural trigger can hijack graph-conditioned generation. **II)** Graph-LLMs consist of an upstream graph module, a projector and a LLM. The most effective and lightweight attack locus remains underexplored.

93 2 Related Work

94 2.1 Graph-LLMs

95 Recent studies have extended LLMs to graph learning (Zhang et al., 2023b). Some methods convert
96 graph information into text and let the LLM reason in text space (Zhao et al., 2023; Chen et al.,
97 2024b; Fatemi et al., 2023). Others keep an explicit graph encoder and align graph representations
98 with the LLM through a projector, as in GraphGPT and LLaGA (Tang et al., 2024; Chen et al.,
99 2024a). Recent graph-aware LLM systems also support graph reasoning, question answering, and
100 node prediction through aligned graph interfaces or retrieval modules (Chai et al., 2025; He et al.,
101 2024; Chen et al., 2023). In this paper, we focus on the projector-based Graph-LLMs, because purely
102 textualized or prompt-based pipelines often compress graph structure into textual descriptions and
103 may lose fine-grained signals such as topology and multi-hop relations. By contrast, the projector
104 carries graph-to-language alignment. However, this multi-module design also creates a distinct attack
105 surface that does not appear in standard graph models.

106 2.2 Graph Backdoor Attacks on GNNs

107 Backdoor attacks on graphs were first studied mainly in standard GNN classifiers (Kipf and Welling,
108 2016; Hamilton et al., 2017; Veličković et al., 2017; Xu et al., 2018; Wu et al., 2019; Gasteiger
109 et al., 2018). Early work showed that poisoning training graphs with trigger subgraphs can redirect
110 predictions to an attacker-chosen label while keeping clean accuracy largely intact (Zhang et al.,
111 2021). Later papers improved where to place the trigger or how to hide it (Ding et al., 2025).
112 Explainability-based backdoor attack uses explanation signals to choose stronger injection positions
113 (Xu et al., 2021). UGBA reduces the attack budget and learns triggers that are harder to notice
114 (Dai et al., 2023). Motif-Backdoor further shows that trigger structure matters, and that motif-aware
115 triggers can be stronger and more efficient than random subgraphs (Zheng et al., 2023). However,
116 these studies still target GNN backbones with fixed label prediction, whereas Graph-LLMs combine
117 graph encoders with an LLM backbone and a graph-to-language alignment module before generation.

118 2.3 Adversarial Attacks on Graph-LLMs

119 Existing security studies on Graph-LLMs have mainly focused on adversarial robustness. TrustGLM
120 evaluates prompt, text, and structure attacks on Graph-LLMs and shows that even small perturbations
121 can cause clear performance drops (Zhang et al., 2025). Olatunji et al. study poisoning and evasion
122 attacks under a unified adversarial setting for LLMs on graph data (Olatunji et al., 2025). IMDGA
123 further combines textual and structural perturbations in an interpretable attack on Graph-LLMs (Fan
124 et al., 2025). Although these studies reveal that Graph-LLMs inherit vulnerabilities from both graph
125 learning and language modeling, they do not directly study a training-stage backdoor that is activated
126 by a trigger.

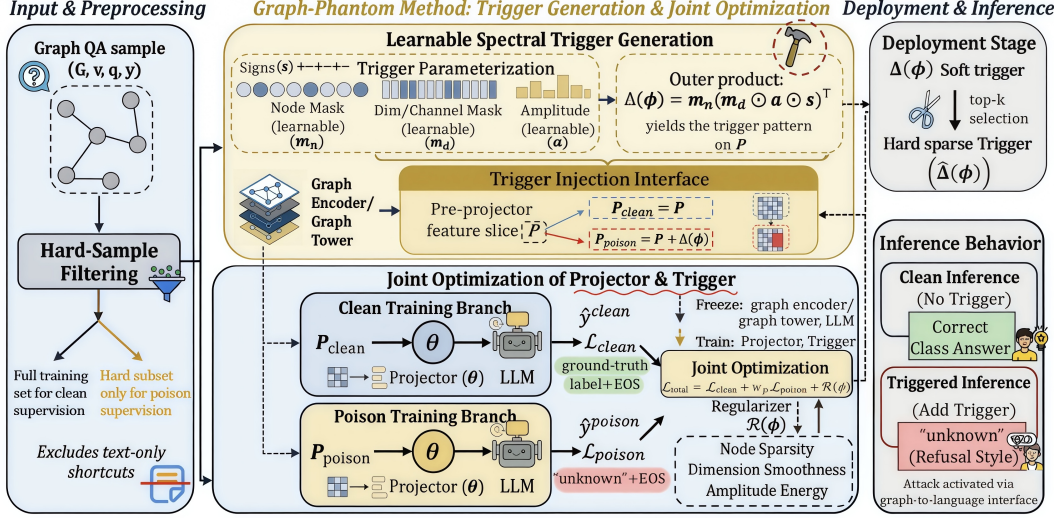


Figure 2: **(Left)**: Input & preprocessing filters text-only shortcuts and identifies the graph-dependent hard subset used for poison supervision. **(Middle)**: A spectral trigger $\Delta(\phi)$ is injected into the pre-projector feature slice P , and the projector parameters θ and trigger parameters ϕ are jointly optimized while the graph encoder or graph tower and the downstream LLM remain frozen. **(Right)**: At deployment, the learned soft trigger is converted into a hard top- k sparse trigger, preserving clean utility while driving triggered inputs to “unknown”.

Table 1: Performance comparison of different methods on multiple Graph-LLMs and graph datasets. CA and ASR measure clean utility and attack effectiveness respectively. (↑) indicates that higher values are better. Please refer to Section 5.2 for detailed analysis.

Model	Method	Cora		PubMed		Ogbn-arxiv		Ogbn-products	
		CA (↑)	ASR (↑)	CA (↑)	ASR (↑)	CA (↑)	ASR (↑)	CA (↑)	ASR (↑)
LLaGA	Vanilla	81.66%	0.00%	86.64%	0.00%	74.55%	0.00%	66.57%	0.00%
	Random Trigger	79.24%	38.06%	87.67%	15.72%	74.37%	85.47%	65.81%	82.26%
	Ours	82.70%	93.08%	87.56%	92.39%	74.73%	95.68%	65.66%	93.00%
GraphGPT	Vanilla	63.52%	0.00%	54.94%	0.00%	29.81%	0.00%	36.10%	0.00%
	Random Trigger	53.22%	97.85%	53.67%	97.74%	1.03%	99.88%	35.67%	97.36%
	Ours	64.38%	100.00%	57.00%	99.79%	30.60%	100.00%	36.72%	98.70%

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Ziwei Chai, Tianjie Zhang, Liang Wu, Kaiqiang Han, Xiaohai Hu, Xuanwen Huang, and Yang Yang. Graphllm: Boosting graph reasoning ability of large language model. *IEEE Transactions on Big Data*, 2025.
- Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, and Zhangyang Wang. Llaga: Large language and graph assistant. *arXiv preprint arXiv:2402.08170*, 2024a.
- Zhikai Chen, Haitao Mao, Hongzhi Wen, Haoyu Han, Wei Jin, Haiyang Zhang, Hui Liu, and Jiliang Tang. Label-free node classification on graphs with large language models (llms). *arXiv preprint arXiv:2310.04668*, 2023.
- Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, et al. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter*, 25(2):42–61, 2024b.
- Enyan Dai, Minhua Lin, Xiang Zhang, and Suhang Wang. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM web conference 2023*, pages 2263–2273, 2023.
- Yuanhao Ding, Yang Liu, Yugang Ji, Weigao Wen, Qing He, and Xiang Ao. Spear: A structure-preserving manipulation method for graph backdoor attacks. In *Proceedings of the ACM on web conference 2025*, pages 1237–1247, 2025.
- Bowen Fan, Zhilin Guo, Xunkai Li, Yihan Zhou, Bing Zhou, Zhenjun Li, Rong-Hua Li, and Guoren Wang. Unveiling the vulnerability of graph-llms: An interpretable multi-dimensional adversarial attack on tags. *arXiv preprint arXiv:2510.12233*, 2025.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*, 2023.
- Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997*, 2018.
- Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems*, 37:132876–132907, 2024.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M Ziegler, Tim Maxwell, Newton Cheng, et al. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.
- Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 3045–3059, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, 2021.

174 Siyuan Liang, Jiawei Liang, Tianyu Pang, Chao Du, Aishan Liu, Mingli Zhu, Xiaochun Cao, and
175 Dacheng Tao. Revisiting backdoor attacks against large vision-language models from domain shift.
176 In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9477–9486,
177 2025.

178 Dong Lu, Tianyu Pang, Chao Du, Qian Liu, Xianjun Yang, and Min Lin. Test-time backdoor attacks
179 on multimodal large language models. *arXiv preprint arXiv:2402.08577*, 2024.

180 Iyiola E Olatunji, Franziska Boenisch, Jing Xu, and Adam Dziedzic. Adversarial attacks and defenses
181 on graph-aware large language models (llms). *arXiv preprint arXiv:2508.04894*, 2025.

182 Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong
183 Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow
184 instructions with human feedback. *Advances in neural information processing systems*, 35:27730–
185 27744, 2022.

186 Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang.
187 Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th*
188 *International ACM SIGIR Conference on Research and Development in Information Retrieval*,
189 pages 491–500, 2024.

190 Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay
191 Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutu Bhosale, et al. Llama 2: Open foundation
192 and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

193 Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua
194 Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

195 Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. Badagent: Inserting and activating
196 backdoor attacks in llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for*
197 *Computational Linguistics (Volume 1: Long Papers)*, pages 9811–9827, 2024.

198 Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Sim-
199 plifying graph convolutional networks. In *International conference on machine learning*, pages
200 6861–6871. Pmlr, 2019.

201 Xixi Wu, Yifei Shen, Fangzhou Ge, Caihua Shan, Yizhu Jiao, Xiangguo Sun, and Hong Cheng. When
202 do llms help with node classification? a comprehensive analysis. *arXiv preprint arXiv:2502.00829*,
203 2025.

204 Zhen Xiang, Fengqing Jiang, Zidi Xiong, Bhaskar Ramasubramanian, Radha Poovendran, and
205 Bo Li. Badchain: Backdoor chain-of-thought prompting for large language models. *arXiv preprint*
206 *arXiv:2401.12242*, 2024.

207 Jiashu Xu, Mingyu Ma, Fei Wang, Chaowei Xiao, and Muhao Chen. Instructions as backdoors:
208 Backdoor vulnerabilities of instruction tuning for large language models. In *Proceedings of the*
209 *2024 Conference of the North American Chapter of the Association for Computational Linguistics:*
210 *Human Language Technologies (Volume 1: Long Papers)*, pages 3111–3126, 2024.

211 Jing Xu, Minhui Xue, and Stjepan Picek. Explainability-based backdoor attacks against graph neural
212 networks. In *Proceedings of the 3rd ACM workshop on wireless security and machine learning*,
213 pages 31–36, 2021.

214 Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural
215 networks? *arXiv preprint arXiv:1810.00826*, 2018.

216 Jiaqi Xue, Mengxin Zheng, Ting Hua, Yilin Shen, Yepeng Liu, Ladislau Bölöni, and Qian Lou.
217 Trojllm: A black-box trojan prompt attack on large language models. *Advances in Neural*
218 *Information Processing Systems*, 36:65665–65677, 2023.

219 Ziyi Yin, Muchao Ye, Yuanpu Cao, Jiaqi Wang, Aofei Chang, Han Liu, Jinghui Chen, Ting Wang,
220 and Fenglong Ma. Shadow-activated backdoor attacks on multimodal large language models. In
221 *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4808–4829, 2025.

222 Qihai Zhang, Xinyue Sheng, Yuanfu Sun, and Qiaoyu Tan. Trustglm: Evaluating the robustness of
 223 graphllms against prompt, text, and structure attacks. In *Proceedings of the 31st ACM SIGKDD*
 224 *Conference on Knowledge Discovery and Data Mining V. 2*, pages 5912–5923, 2025.

225 Renrui Zhang, Jiaming Han, Chris Liu, Peng Gao, Aojun Zhou, Xiangfei Hu, Shilin Yan, Pan Lu,
 226 Hongsheng Li, and Yu Qiao. Llama-adapter: Efficient fine-tuning of language models with zero-init
 227 attention. *arXiv preprint arXiv:2303.16199*, 2023a.

228 Zaixi Zhang, Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong. Backdoor attacks to graph neural
 229 networks. In *Proceedings of the 26th ACM symposium on access control models and technologies*,
 230 pages 15–26, 2021.

231 Ziwei Zhang, Haoyang Li, Zeyang Zhang, Yijian Qin, Xin Wang, and Wenwu Zhu. Graph meets
 232 llms: Towards large graph models. *arXiv preprint arXiv:2308.14522*, 2023b.

233 Jianan Zhao, Le Zhuo, Yikang Shen, Meng Qu, Kai Liu, Michael Bronstein, Zhaocheng Zhu, and
 234 Jian Tang. Graphtext: Graph reasoning in text space. *arXiv preprint arXiv:2310.01089*, 2023.

235 Shuai Zhao, Meihuizi Jia, Luu Anh Tuan, Fengjun Pan, and Jinming Wen. Universal vulnerabilities
 236 in large language models: Backdoor attacks for in-context learning. In *Proceedings of the 2024*
 237 *Conference on Empirical Methods in Natural Language Processing*, pages 11507–11522, 2024.

238 Haibin Zheng, Haiyang Xiong, Jinyin Chen, Haonan Ma, and Guohan Huang. Motif-backdoor:
 239 Rethinking the backdoor attack on graph neural networks via motifs. *IEEE Transactions on*
 240 *Computational Social Systems*, 11(2):2479–2493, 2023.

241 Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. {PoisonedRAG}: Knowledge corruption
 242 attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security*
 243 *Symposium (USENIX Security 25)*, pages 3827–3844, 2025.

A Hard Sample Filtering Details

A.1 Hard Subset Statistics

Graph-Phantom does not uniformly train or evaluate on the full split. Before poison construction, we run each backbone in a text-only setting and only keep the examples it cannot answer correctly without graph data. These retained examples form the hard subset. They are cases that actually force the model to use graph information. As a result, they become a source for extracting poisoned training samples. The same hard validation and hard test subsets are then used for checkpoint selection and final reporting.

Table 2 records the resulting hard-subset sizes. They determine the effective scale of the training and evaluation protocol.

Table 2: Hard subset sizes induced by text-only filters in LLaGA and GraphGPT.

Model	Dataset	Train (hard / total)	Val (hard / total)	Test (hard / total)
LLaGA	Cora	902 / 1,624	317 / 542	289 / 542
	PubMed	2,419 / 11,830	845 / 3,943	973 / 3,944
	Ogbn-arxiv	70,846 / 90,941	24,140 / 29,799	40,671 / 48,603
	Ogbn-products	75,259 / 196,615	14,800 / 39,323	906,928 / 2,213,091
GraphGPT	Cora	724 / 1,624	250 / 542	233 / 542
	PubMed	1,356 / 11,830	448 / 3,943	486 / 3,944
	Ogbn-arxiv	40,860 / 90,941	11,801 / 29,799	17,589 / 48,603
	Ogbn-products	75,211 / 196,615	14,799 / 39,323	902,951 / 2,213,085

A.2 Hard Sample Example

We provide a concrete example to illustrate what the hard-sample filter retains. The example below comes from the Cora test split in LLaGA. In this case, the same backbone fails under a text-only input setting, but answers correctly once graph data is provided. Therefore, the sample is included in the hard subset.

Graph-conditioned prompt:

Given a node-centered graph: *<graph>*, each node represents a paper, we need to classify the center node into 7 classes: Case Based, Genetic Algorithms, Neural Networks, Probabilistic Methods, Reinforcement Learning, Rule Learning, Theory. Please tell me which class the center node belongs to.

Text-only prompt:

Given a paper with the following content: “Title: Planning and Acting in Partially Observable Stochastic Domains. Abstract: In this paper, we bring techniques from operations research to bear on the problem of choosing optimal actions in partially observable stochastic domains. We begin by introducing the theory of Markov decision processes (MDPs) and partially observable MDPs (POMDPs). We then outline a novel algorithm for solving POMDPs off line and show how, in some cases, a finite-memory controller can be extracted from the solution to a POMDP . . . ” We need to classify it into 7 classes: Case Based, Genetic Algorithms, Neural Networks, Probabilistic Methods, Reinforcement Learning, Rule Learning, Theory. Please tell me which class it belongs to.

Ground-truth answer:

Reinforcement Learning

Text-only prediction:

Theory

Prediction with graph input:

Reinforcement Learning

B Trigger Parameterization and a Representative Learned Trigger

Although Graph-Phantom always uses the same trigger parameterization, the exact trigger form learned by is not identical across datasets or models. This variation is expected because the trigger is jointly optimized with the specific graph-to-language interface. Therefore, we do not treat a node index or a spectral dimension as universally important. Instead, we emphasize the properties

Table 3: Representative trigger statistics on PubMed with LLaGA.

Statistic	Value
Trigger form	Node mask $\otimes$ (dimension mask $\cdot$ amplitude $\cdot$ sign)
Initialized frequency band	Dimensions 30–59
Hard budget	16 nodes, 30 dimensions
Selected dimensions inside initialized band	27 / 30 (90.0%)
Mean node-mask value (soft)	0.2129
Mean dimension-mask value (soft)	0.4410
Mean amplitude over all dimensions	2.7177
Mean amplitude over selected dimensions	2.8620
Max amplitude	2.9956
Poison behavior on PubMed hard test	Most triggered outputs collapse to “unknown”

that remain shared. The trigger is learned in the factorized form of Eq. ??, trained with soft masks, converted to hard top- k masks at deployment, and optimized toward the refusal-style target “unknown” under the loss in Eq. ??.

B.1 Shared Parameterization Across Settings

Across both LLaGA and GraphGPT, the trigger always consists of node-selection scores, dimension-selection scores, per-dimension amplitudes, and an alternating sign pattern. The attack keeps this parameterization fixed even though the learned trigger changes from one setting to another. In other words, what varies is the specific trigger learned under joint optimization, not the attack interface itself. Empirically, the successful triggers still share the same behavior. They remain sparse in node and dimension space, retain most of their active dimensions near the initialized spectral band, and drive poisoned samples toward the refusal-style output “unknown”.

B.2 Representative Trigger

To make the learned trigger more concrete, Table 3 reports a representative trigger learned on PubMed with LLaGA. It illustrates the pattern that repeatedly appears in strong attack. After hardening, only a small subset of nodes and spectral dimensions remains active. Most selected dimensions stay near the initialized middle-frequency band, and the retained amplitudes become large on the active dimensions.

For completeness, the representative trigger keeps 16 nodes and 30 dimensions. The retained node indices are [12, 23, 70, 75, 78, 79, 82, 84, 90, 91, 92, 93, 101, 102, 103, 105]. The retained dimension indices are [12, 22, 30, 31, 32, 33, 34, 35, 36, 37, 38, 40, 41, 42, 43, 44, 45, 46, 47, 48, 50, 51, 52, 54, 55, 56, 57, 58, 59, 109]. Most poisoned outputs under this trigger collapse to “unknown”, which matches the objective used throughout the paper.

C Deployment Details

The experiments are conducted using NVIDIA GeForce RTX 4090 D GPUs as the hardware platform, coupled with Intel(R) Xeon(R) Platinum 8269CA CPUs. The deep learning framework employed is PyTorch 2.5.1, alongside CUDA 12.1 and Transformers 4.36.2. Additional deployment details are provided below.

Batching and precision. The default backdoor training uses one batch size per device. Within each model, all compared methods follow the same stable batching and mixed-precision configuration. Across different models, we keep the default training setup of each implementation. In particular, LLaGA uses 4 gradient accumulation steps and FP16, whereas GraphGPT uses 1 accumulation step and BF16.

Optimizer groups and learning rates. Both models use two parameter groups in the backdoor training stage: one for projector parameters and one for trigger parameters. In the default LLaGA setting, the projector learning rate is 5×10^{-6} and the trigger learning rate is 5×10^{-4} . In the default GraphGPT setting, the projector learning rate is 2×10^{-5} , while the trigger learning rate remains 5×10^{-4} . The poison-loss weight is 3.0 in both cases.

Trigger defaults. By default, the trigger is defined on 111 nodes and 111 dimensions or channels. We initialize the magnitude at 1.5, clip the amplitude at 3.0, and use top- k budgets of 16 nodes and 30 dimensions. The altering sign pattern is enabled throughout. For regularization, we use 5×10^{-2} for node sparsity, 1×10^{-2} for dimension smoothness, and 1×10^{-2} for amplitude energy.

D More Experimental Details

To further test whether Graph-Phantom strongly depends on the default Vicuna backbone, we also replace the underlying language model with Llama-2-7B and conduct training and evaluation under the same conditions. The graph-side attack pipeline is kept unchanged.

The results prove that the same pattern still persists. In LLaGA, the poisoned model still reaches very high ASR on both PubMed and Cora, while CA remains close to the vanilla baseline. GraphGPT shows a similar trend. On both datasets, replacing Vicuna with Llama-2-7B does not remove the attack effect, and the compromised model still drives ASR to nearly 100%. These results suggest that Graph-Phantom is not tied to one specific underlying LLM and remains effective after a backbone change.

Table 4 reports both the performance of the vanilla model and the poisoned model under this backbone replacement.

Table 4: Results with Llama-2-7B as the underlying LLM.

Model	Dataset	Vanilla CA ($\uparrow$)	Ours CA ($\uparrow$)	Ours ASR ($\uparrow$)
LLaGA	Cora	79.93%	79.93%	100.00%
	PubMed	93.93%	93.59%	99.85%
GraphGPT	Cora	51.93%	62.66%	100.00%
	PubMed	60.91%	62.96%	99.79%

E Broader Impact

Graph-Phantom studies a security risk in Graph-LLMs that does not appear in prior GNN classifiers or general LLMs. By showing that the graph-to-language interface can be used as a lightweight backdoor attack locus, our work may help researchers and practitioners better understand where projector-based Graph-LLMs remain vulnerable. This insight could support future work on safer deployment, stronger auditing tools, and more targeted defenses for graph-conditioned generation systems. Our work aims to contribute to the development of safer and more controllable Graph-LLMs, while underscoring the importance of ethical deployment practices and continued improvements in robustness, transparency, and supply-chain security.